

Parallel & Distributed Computing

Assignment #1 Report

BLAS Programming, SpMV, Multithreading & Performance Evaluation

Name	Abdullah Rasheed
Student ID	23i-0646
Section	D
Course	Parallel & Distributed Computing, Fall 2026

1. Machine Specification

All development, benchmarking, and profiling was performed on Windows Subsystem for Linux 2 (WSL2), running Ubuntu on a native Windows host. Hardware and software details were collected via **lscpu**, **lstopo-no-graphics**, and standard version queries.

Component	Detail
CPU Model	11th Gen Intel(R) Core(TM) i5-1145G7 @ 2.60GHz
Physical Cores	4
Logical CPUs (threads)	8 (hyperthreading, 2 threads/core)
L1d Cache	192 KiB total (48 KiB per core)
L1i Cache	128 KiB total (32 KiB per core)
L2 Cache	5 MiB total (1.25 MiB per core)
L3 Cache	8 MiB (shared)
RAM (physical, host machine)	16 GB
RAM (allocated to WSL2 VM)	~7.8 GB (WSL2 default allocation; configurable via .wslconfig)
Operating System	Ubuntu on WSL2
Kernel	6.18.33.2-microsoft-standard-WSL2
Compiler	gcc/g++ 15.2.0 (Ubuntu 15.2.0-16ubuntu1)
BLAS Implementation	OpenBLAS 0.3.32
Profiler	perf version 7.0.12 (hardware PMU counters accessible in this WSL2 environment)

Note on hyperthreading: logical CPUs 0-7 map to 4 physical cores with 2 hardware threads each. This is relevant when interpreting thread-count scaling beyond 4 threads, since threads 5-8 share physical execution resources with threads 1-4 rather than running on fully independent cores.

2. Compilation Commands

All source files were compiled with the following flag set, as required by the assignment:

```
gcc -O2 -g -fno-omit-frame-pointer -pthread <file>.c -o <output> [-lopenblas] [-lm]
```

`-lopenblas` was added only for files calling `cblas_*` functions. `-lm` was required for files using `fabs()/fmax()` from `math.h`. Example, for the DGEMM BLAS reference:

```
export OPENBLAS_NUM_THREADS=1
gcc -O2 -g -fno-omit-frame-pointer -pthread 23i-0646-D-TASK3-BLAS.c \
-o 23i-0646-D-TASK3-BLAS -lopenblas -lm
```

OpenBLAS was forced to a single internal thread (via `OPENBLAS_NUM_THREADS=1`) for every BLAS reference comparison, so that the comparison against our own serial (V1) implementations was fair, per the assignment's requirement.

3. Implementation Summary per Task/Version

3.1 Task 1 - DAXPY (BLAS Level 1)

Formula: $y = \alpha * x + y$. Per-element: 1 multiply + 1 add (2 flops), 3 memory operations (read x, read y, write y).

V1 (Serial): a single for-loop over n elements.

V2 (Pthreads): the n-element vector is split into contiguous chunks, one per thread. Each thread writes only to its own disjoint slice of y - no locks required since no two threads ever write the same index. Leftover elements (when n is not divisible by thread count) are distributed one-at-a-time to the first few threads.

V3 (Pthreads + Affinity): identical decomposition to V2, with each thread additionally pinned to a specific logical CPU via `pthread_setaffinity_np()`, called immediately after `pthread_create()` returns a valid handle.

BLAS: `cblas_daxpy(n, alpha, x, 1, y, 1)`, with OpenBLAS forced to 1 thread.

3.2 Task 2 - DGEMV (BLAS Level 2)

Formula: $y = \alpha * A * x + \beta * y$, A is MxN row-major. Per row: N multiply-adds for the dot product with x, plus 1 more for the $\beta * y[i]$ term.

V1 (Serial): nested loop - for each output row i, compute the dot product of A's row i with x.

V2 (Pthreads): the M output rows are split into contiguous blocks across threads. Each thread computes a disjoint set of rows of y, reading its own slice of A and the entire (read-only, safely shared) x vector.

V3 (Pthreads + Affinity): identical row-block decomposition to V2, with blocked CPU pinning (thread t -> CPU t).

BLAS: `cblas_dgemv(CblasRowMajor, CblasNoTrans, M, N, alpha, A, N, x, 1, beta, y, 1)`.

3.3 Task 3 - DGEMM (BLAS Level 3)

Formula: $C = \alpha * A * B + \beta * C$, A is MxK, B is KxN, C is MxN, all row-major. Per output element: a K-length dot product (2K flops); each element of A and B is reused N times across the whole computation, giving DGEMM far higher arithmetic intensity than Level 1/2 kernels.

V1-naive: i-j-k loop order. The innermost k-loop accesses $B[k][j]$ with stride N (a full row jump per iteration) - poor locality for row-major storage.

V1-optimized: i-k-j loop order. The innermost j-loop now accesses both $B[k][j]$ and $C[i][j]$ with stride 1 (contiguous), while $A[i][k]$ is a single scalar reused across the whole inner loop - this is the key change that improves cache locality.

Tiled/Blocked: the i, k, j loops are each split into BLOCK=32 sized tiles, processed tile-by-tile, so the working set of each tile (32x32 doubles = 8KB) fits comfortably inside the 48KB L1d cache per core, maximizing reuse before eviction.

V2 (Pthreads): built on top of the i-k-j (optimized) loop order. The M rows of C are split into contiguous blocks across threads - no locks needed, each thread writes only its own row range.

V3 (Pthreads + Affinity): identical row-block decomposition to V2, with blocked CPU pinning.

BLAS: `cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans, M, N, K, alpha, A, K, B, N, beta, C, N)`.

3.4 Task 4 - SpMV (Sparse Matrix-Vector Multiply)

Dataset: SuiteSparse Williams/webbase-1M, a real web-connectivity graph. $M=N=1,000,005$, $NZ=3,105,536$ (approximately 0.0003% dense).

CSR Construction: the Matrix Market coordinate file is parsed (skipping % comment lines, reading the m/n/nz size line, then row/col/value triples, converting 1-based file indices to 0-based array indices, defaulting missing values to 1.0). CSR arrays (row_ptr, col_idx, values) are built in three passes: (1) count nonzeros per row, (2) prefix-sum into row_ptr, (3) place each entry into its row's slot using a per-row cursor array.

V1 (Serial): for each row i , sum $\text{values}[j] \times x[\text{col_idx}[j]]$ for j across that row's nonzero range $[\text{row_ptr}[i], \text{row_ptr}[i+1])$.

V2 (Pthreads): rows are split evenly by ROW COUNT (not by nonzero count) across threads - the same simple decomposition used for DGEMV. Each thread additionally tallies how many nonzeros it actually processed, to make load imbalance directly measurable.

V3 (Pthreads + Affinity): identical row decomposition to V2, with blocked CPU pinning.

A checksum (sum of all $y[i]$) is printed with 6 decimal digits after every run, used to confirm V1/V2/V3 all produce the identical mathematical result.

4. Scheduling Strategy and CPU Affinity

Strategy used: BLOCKED mapping (thread t is pinned to logical CPU t , wrapping modulo the number of online CPUs).

Justification: every kernel in this assignment decomposes work into contiguous, disjoint blocks (vector chunks for DAXPY, row blocks for DGEMV/DGEMM/SpMV) - there is already a natural 1-to-1 mapping between threads and contiguous work units. A blocked thread-to-CPU mapping is the simplest, most predictable choice consistent with this decomposition, and this machine has enough logical CPUs (8) to give every thread used in this assignment (up to 8) a distinct core, avoiding unnecessary sharing.

Affinity was set using `pthread_setaffinity_np()` immediately after each `pthread_create()` call, since a valid `pthread_t` handle only exists once `pthread_create()` returns it - affinity cannot be set before the thread exists.

CPU mapping was verified directly in program output for every V3 run (e.g. Task 1 V3: `thread0->CPU0 thread1->CPU1 thread2->CPU2 . . .`), confirming the pinning worked as intended in every task.

Important finding: across all four kernels, CPU affinity did not automatically produce large or consistent speedups over plain pthreads (V2). In Tasks 1 and 4, V3 was sometimes measurably slower than V2 due to the added cost of the `pthread_setaffinity_np()` system call itself. In Task 2 and 3, V3 gave a small but real improvement, most notably smoothing out an anomalous slowdown V2 showed at 4 threads for DGEMM (V2: 29.2ms vs V3: 23.2ms at size=500, 4 threads) - consistent with affinity reducing scheduling-induced variability rather than fundamentally changing throughput. This matches the assignment's explicit guidance that affinity should not be assumed to automatically improve performance.

5. Correctness Validation

Two layers of correctness checking were used for every kernel and every version:

- **Hand-computable fixed case:** a small, manually-verifiable input (e.g. DAXPY: $\alpha=2.5$, $x=[1,2,3]$, $y=[10,10,10] \rightarrow$ expected $y=[12.5,15.0,17.5]$; DGEMM: a 2x2 case verified by hand matrix multiplication) run through every version, compared against the exact expected output.
- **Validation against serial on real data:** each parallel version (V2, V3) and each BLAS reference was run on the actual benchmark-sized random data and compared element-by-element against the plain serial (V1) result, for every thread count used.

Tolerance: a 1e-12 absolute tolerance was used for comparisons where the exact same summation order is used (e.g. our own V1 vs V2/V3, since the loop structure is identical and floating-point results are effectively bit-identical). A 1e-9 *relative* tolerance was used specifically when comparing against OpenBLAS or against a reordered-loop version (e.g. DGEMM's i-j-k vs i-k-j), since these use genuinely different summation orders internally - floating-point addition is not associative, so two independently-correct implementations can differ by more than a tiny absolute amount purely from accumulated rounding differences, especially for larger reductions (thousands of terms). This was confirmed directly: an initial 1e-12 absolute-tolerance comparison of our DGEMV against `cblas_dgemv()` at size=2000 failed, even though both results were mathematically correct - switching to a 1e-9 relative tolerance resolved this correctly.

Result: every correctness check across all 4 tasks and every version (V1, V2, V3, BLAS, and DGEMM's naive/optimized/tiled variants) returned PASS. For SpMV specifically, V1/V2/V3 additionally produced an identical checksum (5501349.631742) at every thread count tested (1, 2, 4, 8), directly confirming the parallel row-decomposition and the underlying CSR parser are correct.

6. Timing Results (Average and Minimum over 5 Trials)

All timings below use `clock_gettime(CLOCK_MONOTONIC, ...)` and are reported in milliseconds, averaged and minimum over 5 trials per configuration, per the assignment's requirement.

6.1 Task 1 - DAXPY (n=30000)

Version	Threads	Avg (ms)	Min (ms)
V1 (Serial)	1	0.0102	0.0068
V2	1	0.2859	0.1943
V2	2	0.3658	0.2691
V2	4	0.3232	0.2539
V2	8	0.7405	0.3983
V3	1	0.3169	0.1501
V3	2	0.4435	0.4032
V3	4	1.2922	0.7493
V3	8	1.0970	0.6952
BLAS (cblas_daxpy)	1	0.0055	0.0051

6.2 Task 2 - DGEMV (size=2000x2000)

Version	Threads	Avg (ms)	Min (ms)
V1 (Serial)	1	5.8815	4.4053
V2	1	6.2895	6.2121
V2	2	3.9600	3.7361
V2	4	2.9401	2.7194
V2	8	2.8818	2.2860
V3	1	5.9966	4.8183
V3	2	3.3257	2.9174
V3	4	2.4856	1.9961
V3	8	2.2289	1.9800
BLAS (cblas_dgemv)	1	2.0857	1.6050

6.3 Task 3 - DGEMM (size=500x500x500)

Version	Threads	Avg (ms)	Min (ms)
V1-naive (i-j-k)	1	126.6646	120.2521
V1-optimized (i-k-j)	1	69.4410	59.1053
Tiled/Blocked	1	47.9698	45.1394
V2	1	65.4717	54.6819

Version	Threads	Avg (ms)	Min (ms)
V2	2	36.8214	28.5624
V2	4	40.9224	29.1797
V2	8	19.9768	18.9223
V3	1	63.5245	58.8941
V3	2	35.6385	33.6487
V3	4	26.0569	23.2042
V3	8	19.8316	18.6093
BLAS (cblas_dgemm)	1	5.4536	4.4262

6.4 Task 4 - SpMV (webbase-1M, NZ=3,105,536)

Version	Threads	Avg (ms)	Min (ms)
V1 (Serial)	1	7.1238	4.9679
V2	1	10.1308	7.8952
V2	2	9.5643	6.0159
V2	4	9.1557	4.7592
V2	8	4.3316	3.9428
V3	1	8.2674	6.3067
V3	2	6.8923	5.8262
V3	4	5.1330	4.6321
V3	8	4.8814	4.0007

7. Speedup and Efficiency Plots

Speedup is computed relative to each kernel's V1 serial minimum time. Efficiency = speedup / thread count.

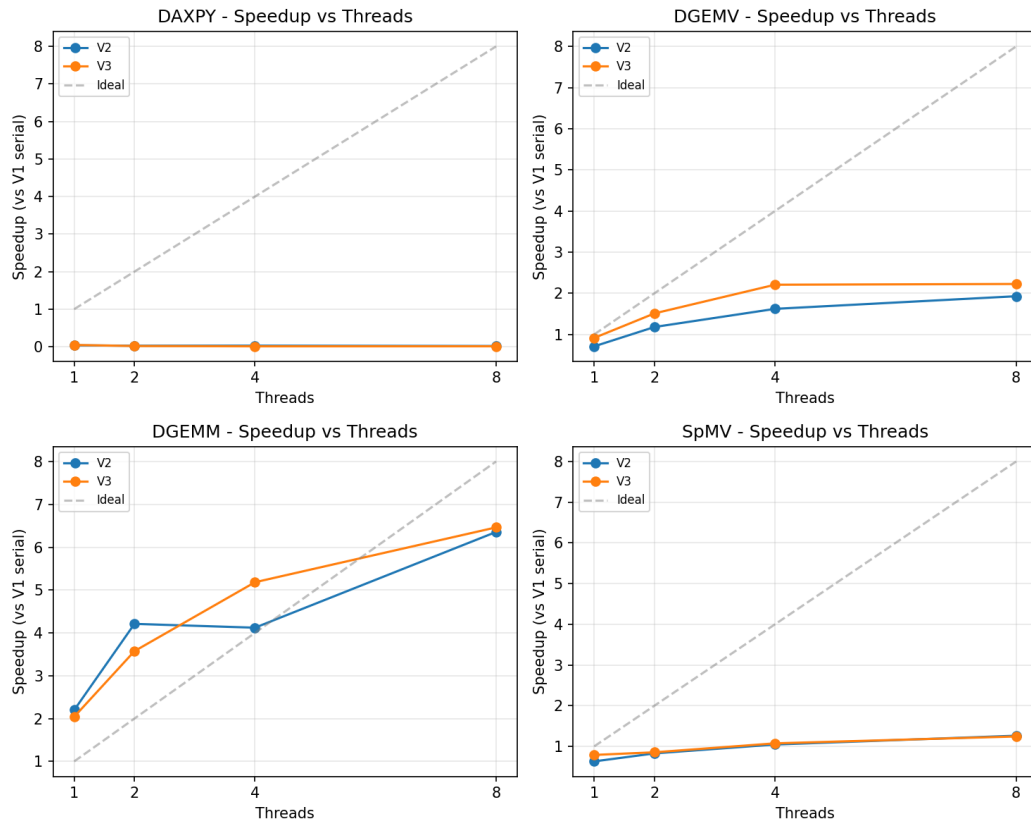


Figure 1: Speedup vs thread count for all four kernels, V2 and V3.

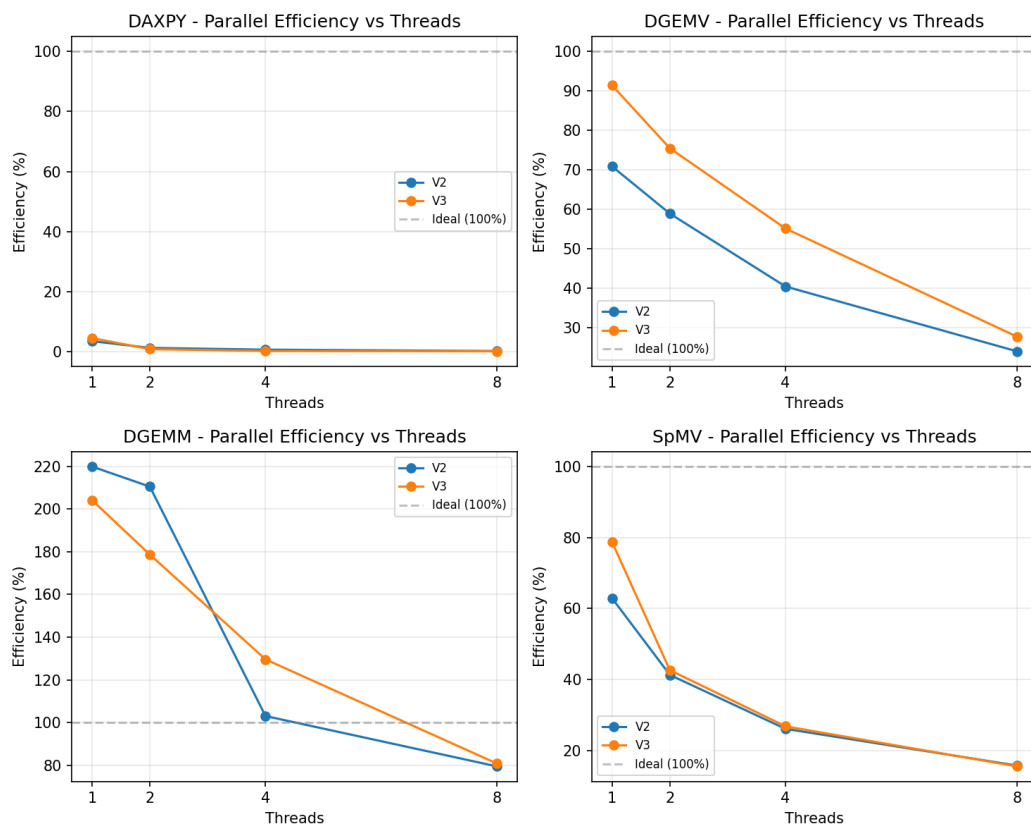


Figure 2: Parallel efficiency vs thread count for all four kernels, V2 and V3.

DAXPY shows negative speedup at every thread count (0.02x-0.04x) - threading is actively harmful here, since the workload is too small and too memory-bandwidth-bound to amortize thread creation/join/affinity overhead.

DGEMV and DGEMM both show a clean, expected pattern: speedup increases with thread count while efficiency steadily declines - classic diminishing returns as synchronization and shared-resource (cache/memory-bandwidth) contention grow with more threads.

SpmV shows the weakest scaling of the three parallelizable kernels (max ~1.26x speedup at 8 threads for V2), directly attributable to load imbalance (see Section 9).

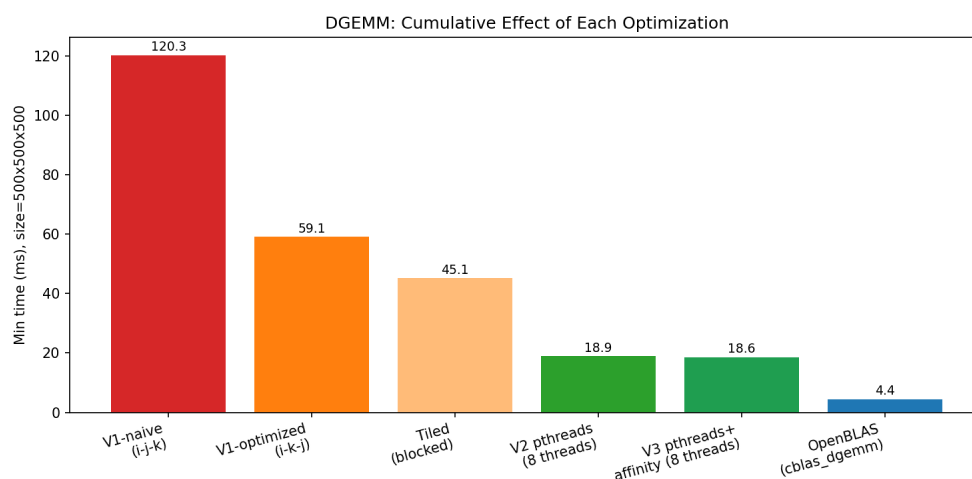


Figure 3: Cumulative effect of each optimization applied to DGEMM, size=500. Loop reordering, tiling, threading, and vendor BLAS each contribute a measurable improvement, stacking to a total ~27x speedup from naive serial to OpenBLAS.

8. Perf Evidence and Interpretation

perf stat was run on the V1 (or V1-optimized, for DGEMM) binary of each kernel, capturing task-clock, cycles, instructions, cache-references, and cache-misses. IPC (instructions/cycle) and cache-miss rate (misses/references) were computed from the raw counters.

Kernel	Task-clock (ms)	Instructions	IPC	Cache-miss rate
DAXPY (V1)	1.38	4,683,185	2.37	56.96%
DGEMV (V1)	83.30	380,596,618	1.92	54.71%
DGEMM (V1-opt)	475.01	7,012,662,017	3.75	0.86%
SpMV (V1)	902.40	10,227,936,005	3.25	45.30%

DGEMM's cache-miss rate (0.86%) is 50-65x lower than the other three kernels, and it shows the best IPC - direct, measured confirmation of the arithmetic-intensity prediction from Section 9: DGEMM's high data reuse keeps its working set resident in cache, so the CPU rarely stalls waiting on memory. DAXPY, DGEMV, and SpMV all cluster together with cache-miss rates between 45-57%, consistent with all three being memory-bound.

Honest caveat: DAXPY's measured IPC (2.37) is higher than DGEMV's (1.92), despite DAXPY having the lower arithmetic intensity of the two - superficially this looks inconsistent with the model. The likely explanation is measurement reliability, not a wrong theory: DAXPY's entire program (correctness check plus both benchmark sizes, all trials) completed in only 1.38ms of CPU time, generating just 10,906 cache-reference samples - far too few for a statistically stable reading, compared to DGEMV's 2.47 million samples or DGEMM's 109.7 million. This limitation is stated explicitly here rather than forcing the small-sample result to fit the model.

9. Arithmetic Intensity and Locality Analysis

Using the operation-count model supplied in the assignment (Figure 1), arithmetic intensity (FLOPs per byte moved) was estimated for the representative size used in each kernel's benchmark:

Kernel	Flops	Bytes moved	Arithmetic Intensity
DAXPY (n=30000)	60,000	720,000	0.083 flops/byte
SpMV (nnz=3.1M)	6,211,072	~53,266,512	0.117 flops/byte (approx.)
DGEMV (n=2000)	8,002,000	32,048,000	0.250 flops/byte
DGEMM (n=500)	250,250,000	8,000,000	31.281 flops/byte

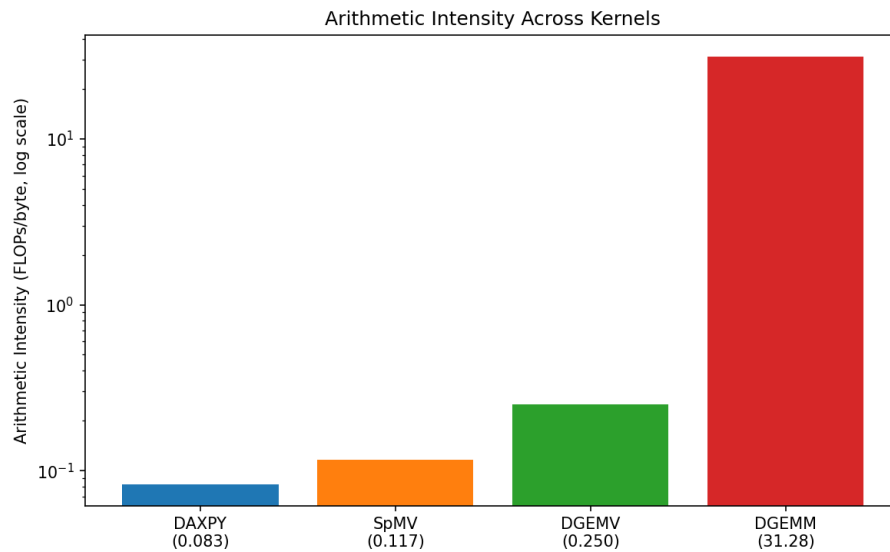


Figure 4: Arithmetic intensity comparison, log scale. DGEMM's AI exceeds the other three kernels by more than two orders of magnitude.

Key insight: DGEMM's AI (31.28) is over 100x higher than DAXPY, DGEMV, or SpMV, because DGEMM reuses each element of A and B up to N times (once per output element that needs it), whereas the other three kernels touch each input element exactly once. This is precisely why loop reordering (i-j-k $\rightarrow$ i-k-j) and cache tiling produced large, measurable speedups for DGEMM (2.0x and a further 1.3x respectively) but would offer little to no benefit for DAXPY, DGEMV, or SpMV - there is no reuse in those kernels for a cache-blocking strategy to exploit. DGEMM's AI also *grows* with matrix size (more reuse opportunity as N increases), while the other three kernels' AI stays roughly constant regardless of size - a fundamental structural difference between BLAS Level 3 and everything below it.

10. Comparing BLAS Level 1, Level 2, Level 3, and SpMV

DAXPY (Level 1) is a pure vector-vector operation with the lowest arithmetic intensity of any kernel tested (0.083 flops/byte) and completely predictable, sequential memory access. It is unambiguously memory-bandwidth-bound: its total useful work is so small that thread-creation overhead alone made every parallel version slower than plain serial execution.

DGEMV (Level 2) has moderate arithmetic intensity (0.250 flops/byte) - each matrix element is read exactly once, but the vector x is reused across every row. This gives it enough real work per thread to benefit from parallelization (up to ~2.2x speedup at 8 threads), unlike DAXPY, while still remaining fundamentally

memory-bound - OpenBLAS still comfortably outperforms even the best hand-written pthreads version.

DGEMM (Level 3) is the clear outlier: its arithmetic intensity (31.28 flops/byte) is over 100x higher than the other kernels because A and B are reused N times each. This reuse is exploitable via loop reordering and cache blocking, both of which produced measurable, real speedups purely from improving locality, before any threading was applied at all. DGEMM is compute-bound rather than memory-bound.

SpMV sits in an unusual position: its estimated arithmetic intensity (0.117) is actually similar to DAXPY's, but its access pattern is *irregular* rather than sequential - `x[col_idx[j]]` jumps unpredictably based on the sparsity structure of the underlying graph, defeating hardware prefetching. Despite performing fewer total FLOPs than DGEMV at the tested sizes (6.2M vs 8.0M), SpMV's minimum time (4.97ms) was comparable to or worse than DGEMV's (4.41ms) - concrete evidence that irregular memory access costs substantially more per FLOP than dense, predictable access, even when the total arithmetic work is lower.

11. SpMV Load Imbalance Analysis

Row-block decomposition splits rows evenly by COUNT, not by actual work. Since webbase-1M is a real web-connectivity graph, individual rows vary enormously in nonzero count (some pages/nodes have very few links, others - hubs - have thousands). This was measured directly by having each thread report its own total nonzero count processed:

Threads	Min nnz (thread)	Max nnz (thread)	Imbalance Ratio
1	3,105,536	3,105,536	1.00x
2	1,441,749	1,663,787	1.15x
4	551,988	1,072,417	1.94x
8	264,652	760,934	2.88x

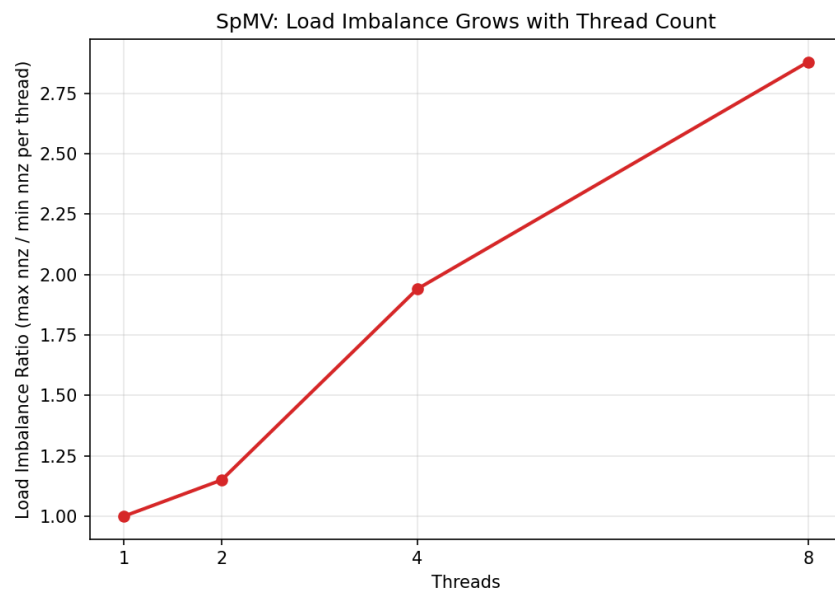


Figure 5: Load imbalance ratio increases sharply as thread count grows.

At 8 threads, the most-loaded thread handles nearly 3x the nonzeros of the least-loaded thread. This directly explains SpMV's poor observed scaling (max ~1.26x speedup at 8 threads, versus DGEMV's ~2.2x or DGEMM's ~6.5x): the busiest thread becomes the bottleneck at every `pthread_join()`, while lighter-loaded threads sit idle waiting. This differs fundamentally from a dense row-block workload (as in DGEMV), where every row has exactly N elements and is guaranteed to be perfectly balanced by construction. CPU affinity (V3) did not change this imbalance at all - the ratios were identical between V2 and V3 at every thread count, confirming that affinity only controls *where* threads run, not *what work* they are assigned.

12. Bottleneck Identification and Proposed Optimizations

Kernel	Primary Bottleneck	Proposed Next Optimization
DAXPY	Thread creation/join overhead vastly exceeds useful work; memory bandwidth for any real workload size	Use a persistent thread pool (create threads once, reuse across many DAXPY calls) instead of creating/destroying threads per call
DGEMV	Memory bandwidth - each matrix element read exactly once, no reuse to exploit	Explore blocked/tiled DGEMV to improve reuse of x across cache lines, though gains would likely be modest given Level 2's inherent memory-bound nature
DGEMM	At high thread counts, shared L2/L3 cache contention and memory bandwidth begin to cap scaling; still far short of OpenBLAS's register-level tiling and SIMD	Implement register-blocking / micro-kernel tiling (compute a small sub-block of C entirely in registers before writing back), the technique that gives vendor BLAS its remaining advantage
SpMV	Load imbalance from uneven per-row nonzero counts; irregular memory access defeating prefetch	Switch from row-count-based decomposition to nonzero-count-based decomposition (assign each thread an equal share of total nonzeros, not rows), which would directly address the measured 2.88x imbalance at 8 threads

Final Discussion

Across all four kernels, three consistent themes emerged. **First**, arithmetic intensity - not thread count - is the dominant predictor of how much a kernel benefits from either cache optimization or parallelization: DGEMM's high reuse made loop reordering and tiling worthwhile, while the memory-bound kernels (DAXPY, DGEMV, SpMV) saw little to no benefit from such techniques. **Second**, more threads is not automatically better: DAXPY's workload was too small to ever benefit from threading, and even DGEMM's speedup growth from 4 to 8 threads was noticeably sub-linear, reflecting real hardware limits (shared cache, memory bandwidth, and - for this 4-physical-core/8-logical-CPU machine - hyperthreading contention beyond 4 threads). **Third**, CPU affinity is a tool for reducing measurement variability and scheduling noise, not a guaranteed performance win - it helped modestly in Tasks 2 and 3, was neutral-to-harmful in Task 1, and had zero effect on SpMV's load imbalance, exactly as the assignment's guidance anticipated. Finally, OpenBLAS outperformed even the best hand-written, affinity-pinned pthreads implementation in every single task, by margins ranging from roughly 1.3x (DAXPY, where thread overhead dominated our own results) to over 27x (DGEMM) - underscoring that vendor libraries combine algorithmic techniques (blocking, SIMD, micro-kernels) that go well beyond what straightforward loop parallelization can achieve on its own.